

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN –
ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN
ỨNG DỤNG QUẢN LÝ NHÀ XE

Môn học: Cấu trúc dữ liệu và Giải thuật
Giảng viên: Nguyễn Thanh Tình
Lớp: 25CTT4A

STT	Họ tên	MSSV
1	Mai Duy An	25120276
2	Ngô Nguyễn Gia An	25120277

Thành phố Hồ Chí Minh, 2026

Lời cảm ơn

Nhóm thực hiện xin chân thành cảm ơn giảng viên Nguyễn Thanh Tình đã hướng dẫn tận tình và góp ý trong suốt quá trình xây dựng đồ án. Những yêu cầu về thiết kế, cấu trúc dữ liệu, kiểm thử và trình bày báo cáo đã giúp nhóm hoàn thiện sản phẩm theo hướng rõ ràng, có thể kiểm chứng và bám sát nội dung môn học.

Mục lục

Lời cảm ơn	1
1 Giới thiệu	6
1.1 Bối cảnh	6
1.2 Mục tiêu	6
1.3 Phạm vi	6
1.4 Giới hạn thực tế	6
2 Phân tích yêu cầu	8
2.1 Người dùng	8
2.2 Xe vào	8
2.3 Xe ra	8
2.4 Bản đồ và mã ô	8
2.5 Tính phí, hóa đơn và blacklist	9
2.6 Undo	9
3 Thiết kế hệ thống	10
3.1 Kiến trúc tổng thể	10
3.2 Sơ đồ lớp mức thiết kế	11
3.3 Ánh xạ yêu cầu với mã nguồn	11
3.4 Luồng xe vào	14
3.5 Luồng xe ra	15
3.6 Luồng hóa đơn và blacklist	16
3.7 Luồng undo	17
4 Cấu trúc dữ liệu và giải thuật	18
4.1 Các cấu trúc dữ liệu	18
4.2 Algorithm	19
4.3 BFS tìm ô gần nhất	19
4.4 Bảng độ phức tạp	20
5 Thiết kế và lưu trữ dữ liệu	21
5.1 Nguyên tắc chung	21
5.2 DanhSachSV.txt	21
5.3 map.txt	21
5.4 active_sessions.txt	22
5.5 history.txt	22

5.6	monthly_usage.txt, monthly_bills.txt, revenue.txt	22
6	Cài đặt và hướng dẫn chạy	23
6.1	Linux TUI	23
6.2	Windows TUI	23
6.3	Windows GUI	23
6.4	Chạy kiểm thử backend	24
7	Giao diện và chức năng	25
8	Kiểm thử	27
8.1	Kết quả build	27
8.2	Bảng test case	27
9	Đánh giá và hướng phát triển	32
9.1	Những phần đã hoàn thành	32
9.2	Điểm mạnh	32
9.3	Hạn chế	32
9.4	Hướng phát triển	33
	Kết luận	34
A	Cấu trúc repository đã kiểm tra	36
B	Lệnh kiểm tra đã chạy trong môi trường hiện tại	37

Danh sách hình vẽ

3.1	Kiến trúc hệ thống	10
3.2	Sơ đồ lớp chính	11
3.3	Luồng xử lý xe vào	14
3.4	Luồng xử lý xe ra	15
3.5	Luồng hóa đơn và blacklist	16
3.6	Luồng undo	17

Danh sách bảng

3.1	Ánh xạ yêu cầu với mã nguồn	12
4.1	Cấu trúc dữ liệu trong repository	18
4.2	Độ phức tạp các thao tác nghiệp vụ chính	20
5.1	Định dạng <code>DanhSachSV.txt</code> (Bảng 5.1)	21
5.2	Định dạng <code>active_sessions.txt</code> (Bảng 5.2)	22
5.3	Các file thống kê và hóa đơn (Bảng 5.3)	22
7.1	Giao diện và backend liên quan	25
8.1	Kết quả build	27
8.2	Test case chi tiết	27

Chương 1

Giới thiệu

1.1 Bối cảnh

Quản lý nhà xe trong môi trường trường học cần theo dõi người gửi xe, biển số, vị trí đỗ, lượt gửi, lịch sử, hóa đơn và tình trạng thanh toán. Nếu chỉ ghi nhận thủ công, việc tra cứu xe đang gửi, tính phí theo tháng và xử lý người dùng nợ quá hạn rất dễ sai lệch.

1.2 Mục tiêu

Đề án xây dựng ứng dụng C++ quản lý nhà xe với các mục tiêu chính:

- Quản lý người dùng theo ID, họ tên, biển số, phân loại và tổng lượt gửi xe.
- Ghi nhận xe vào, xe ra, vị trí đỗ và lịch sử.
- Tìm ô đỗ phù hợp gần cổng nhất dựa trên bản đồ 2D (BFS).
- Tính lượt gửi xe, phí, thống kê tháng, hóa đơn, thanh toán và blacklist.
- Cung cấp hai lớp giao diện: TUI đa nền tảng và GUI Win32 trên Windows.
- Hỗ trợ undo thao tác xe vào/ra trong phiên chạy hiện tại.

1.3 Phạm vi

Repository có TUI trong `app/main_tui.cpp` and GUI Win32 trong `app/main.cpp`. Hai giao diện dùng chung backend qua các wrapper `app/models.hpp`, `app/ParkingMap.hpp`, `app/ParkingSystem.hpp`; implementation thực tế nằm trong các file `_tui.hpp` tương ứng. Dữ liệu lưu bằng file văn bản UTF-8, phân tách trường bằng ký tự `|`.

1.4 Giới hạn thực tế

- Undo chỉ tồn tại trong bộ nhớ của phiên chạy hiện tại, không được lưu ra đĩa.
- GUI Win32 cần Windows hoặc MSYS2 để build và chạy thực tế.

- `revenue.txt` được giữ như dữ liệu legacy, không phải nguồn sự thật chính của doanh thu.

Chương 2

Phân tích yêu cầu

2.1 Người dùng

Mỗi người dùng gồm `id`, `name`, `licensePlate`, `role` và `totalParkingUnits`. Phân loại được định nghĩa trong `UserRole`: 0 là sinh viên, 1 là giảng viên, 2 là khách ngoài trường. Backend hỗ trợ đăng ký (kiểm tra trống, ký tự l, trùng ID, trùng biển số), tra cứu theo ID, duyệt danh sách và xóa người dùng nếu không còn xe đang gửi hoặc hóa đơn chưa thanh toán.

2.2 Xe vào

Luồng xe vào chỉ yêu cầu nhập ID. Hàm `ParkingSystem::vehicleEntry(...)` kiểm tra người dùng tồn tại, xe chưa ở trong bãi, không bị blacklist, tìm ô gần nhất phù hợp, tạo `ActiveSession`, ghi lịch sử `VAO`, lưu `active_sessions.txt` và đưa thao tác vào stack undo.

2.3 Xe ra

Luồng xe ra gồm nhập ID lần đầu để xem vị trí qua `previewVehicleExit(...)` and nhập lại ID để xác nhận qua `vehicleExit(...)`. Backend tính thời lượng, số lượt, tiền, cập nhật `DanhSachSV.txt`, `monthly_usage.txt`, giải phóng ô, ghi lịch sử `RA` và hỗ trợ undo.

2.4 Bản đồ và mã ô

Bản đồ đọc từ `map.txt`. Ký tự `#` là tường, `.` là lối đi, `E` là cổng, `P` là ô sinh viên hoặc khách, `T` là ô giảng viên, `X` chỉ xuất hiện khi hiển thị trạng thái đang có xe. Khu lớn dùng các chữ `A-K` (bỏ `J`); khu `A` gần cổng nhất. Mã ô do `ParkingMap::makeSlotCode(...)` tạo, ví dụ `A101`, `B208`.

2.5 Tính phí, hóa đơn và blacklist

Công thức số lượt nằm trong `calculateParkingUnits(...)`: mỗi tối đa 10 giờ tính một lượt, trên 10 giờ tính thêm lượt (làm tròn lên). Phí được khai báo trong `ParkingSystem`: sinh viên 3.000 đồng/lượt, giảng viên 5.000 đồng/lượt, khách 5.000 đồng/lượt. Hóa đơn tháng được tạo bởi `processMonthlyBilling(...)` cho các tháng đã qua mà chưa có bill. Hóa đơn quá hạn bị phạt 10.000 đồng/ngày và làm người dùng bị blacklist cho đến khi không còn hóa đơn quá hạn chưa thanh toán.

2.6 Undo

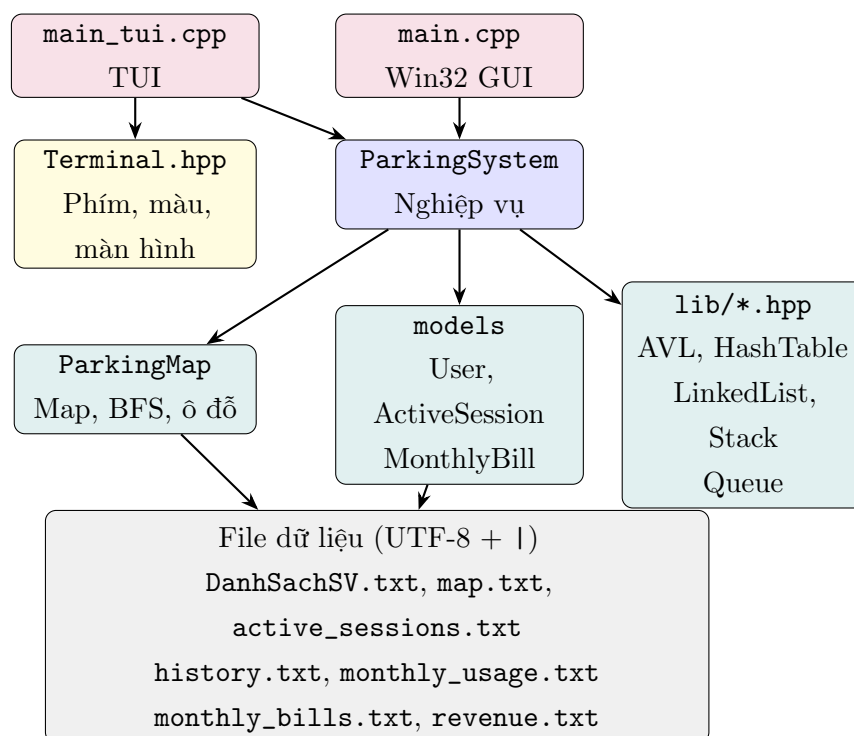
Undo được cài bằng `lib::Stack<AdminAction>`. Backend hỗ trợ undo xe vào (xóa session, thả ô) và xe ra (khôi phục session, trừ thống kê). Undo xe ra bị từ chối nếu người dùng đã có phiên gửi mới, ô cũ không còn trống, người dùng không còn tồn tại, hóa đơn tháng đã được tạo hoặc dữ liệu thống kê không nhất quán.

Chương 3

Thiết kế hệ thống

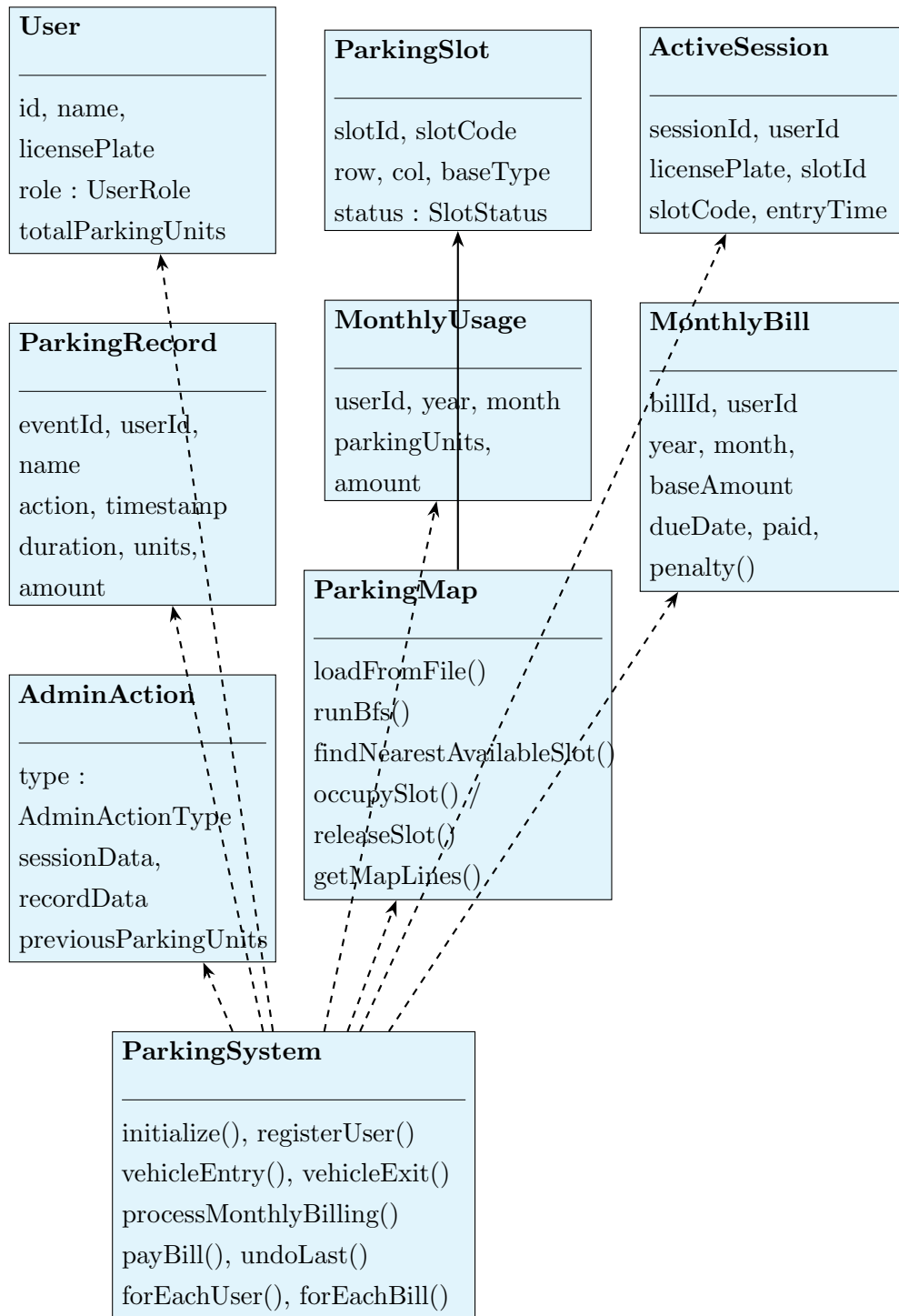
3.1 Kiến trúc tổng thể

Kiến trúc thực tế là kiến trúc nhiều lớp. TUI và GUI là lớp trình bày; `ParkingSystem` là lớp nghiệp vụ; `ParkingMap`, models và các cấu trúc dữ liệu hỗ trợ xử lý; các file `.txt` là lớp lưu trữ. TUI và GUI dùng chung backend, không có hai `ParkingSystem` độc lập.



Hình 3.1: Kiến trúc hệ thống

3.2 Sơ đồ lớp mức thiết kế



Hình 3.2: Sơ đồ lớp chính

3.3 Ánh xạ yêu cầu với mã nguồn

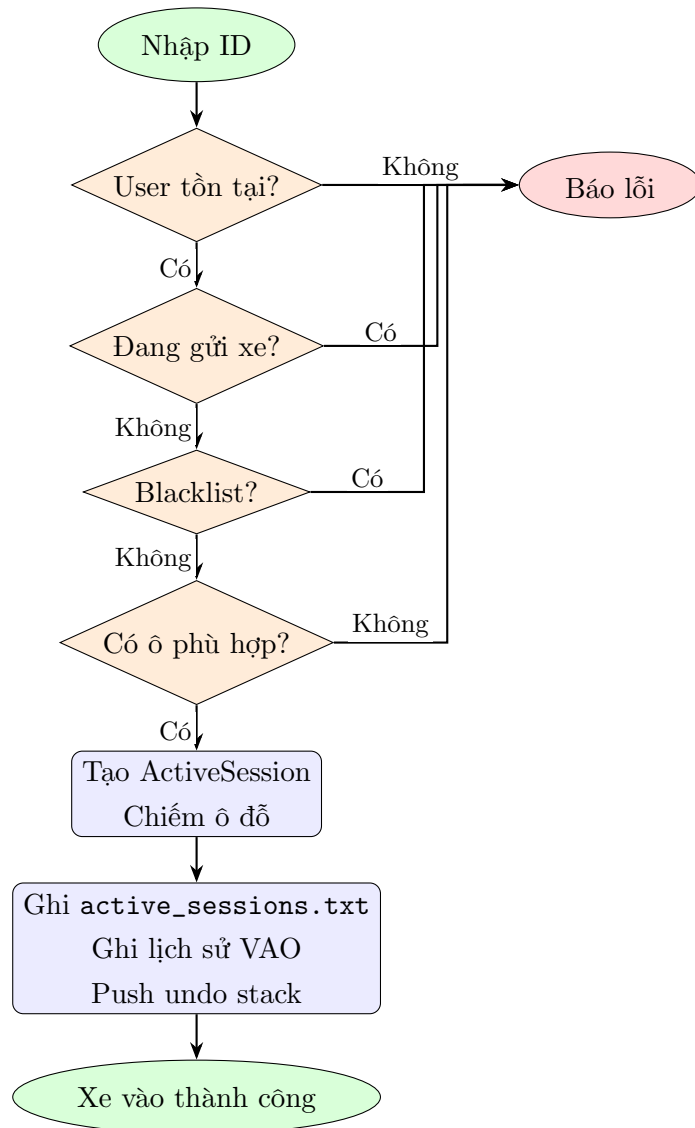
Bảng 3.1: Ánh xạ yêu cầu với mã nguồn

Yêu cầu	File	Class/hàm	Vai trò
Model người dùng	app/models_tui.hpp	User, UserRole	Lưu ID, họ tên, biển số, role, tổng lượt.
Đăng ký	app/ ParkingSystem_tui.hpp	registerUser(...)	Kiểm tra rỗng, ký tự , trùng ID, trùng biển số, ghi file.
Tra cứu	app/ ParkingSystem_tui.hpp	findUser(...), getUserList()	Tìm user và tạo dữ liệu hiển thị.
Xe vào	app/ ParkingSystem_tui.hpp	vehicleEntry(...)	Kiểm tra điều kiện, chọn ô, tạo phiên, ghi lịch sử, push undo.
Xem vị trí xe ra	app/ ParkingSystem_tui.hpp	previewVehicleExit(...)	Lấy <code>ActiveSession</code> theo ID.
Xe ra	app/ ParkingSystem_tui.hpp	vehicleExit(...)	Xác nhận ID, tính lượt và tiền, cập nhật thống kê, giải phóng ô.
Bản đồ	app/ParkingMap_tui.hpp	loadFromFile(...), getMapLines()	Đọc map gốc và hiển thị trạng thái X.
Tìm ô gần nhất	app/ParkingMap_tui.hpp	runBfs(), findNearest AvailableSlot(...)	BFS từ cổng, lọc loại ô, phá hòa theo slotCode.
Hóa đơn	app/ ParkingSystem_tui.hpp	processMonthly Billing(...), forEachBill(...)	Tạo và duyệt hóa đơn tháng.
Thanh toán	app/ ParkingSystem_tui.hpp	payBill(...)	Cập nhật trạng thái paid, ghi lịch sử <code>THANH_TOAN</code> .
Blacklist	app/ ParkingSystem_tui.hpp	isBlacklisted(...), forEachBlacklisted(...)	Chặn xe vào khi có hóa đơn quá hạn.

Bảng 3.1 (tiếp theo)

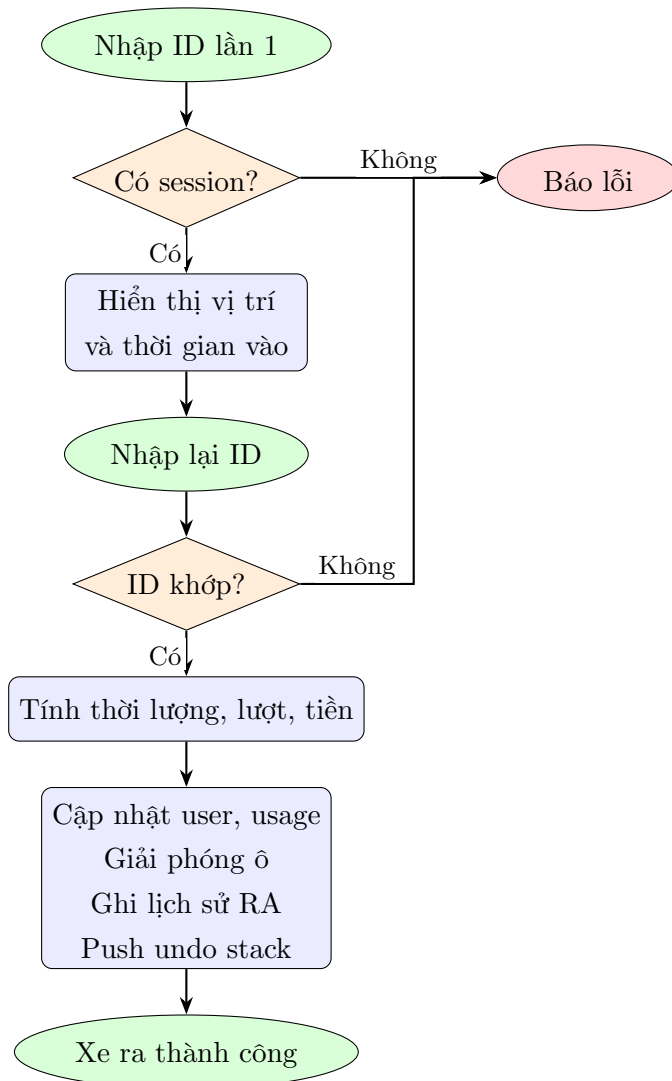
Yêu cầu	File	Class/hàm	Vai trò
Undo	app/ ParkingSystem_tui.hpp	undoLast(...), restoreExitState(...)	Hoàn tác xe vào/ra trong phiên chạy.
TUI	app/main_tui.cpp	runMainMenu(...), ...Screen	Giao diện terminal đa nền tảng.
GUI	app/main.cpp	handleEntry(), handleExit(), handlePayBill()	Giao diện Win32 gọi backend chung.

3.4 Luồng xe vào



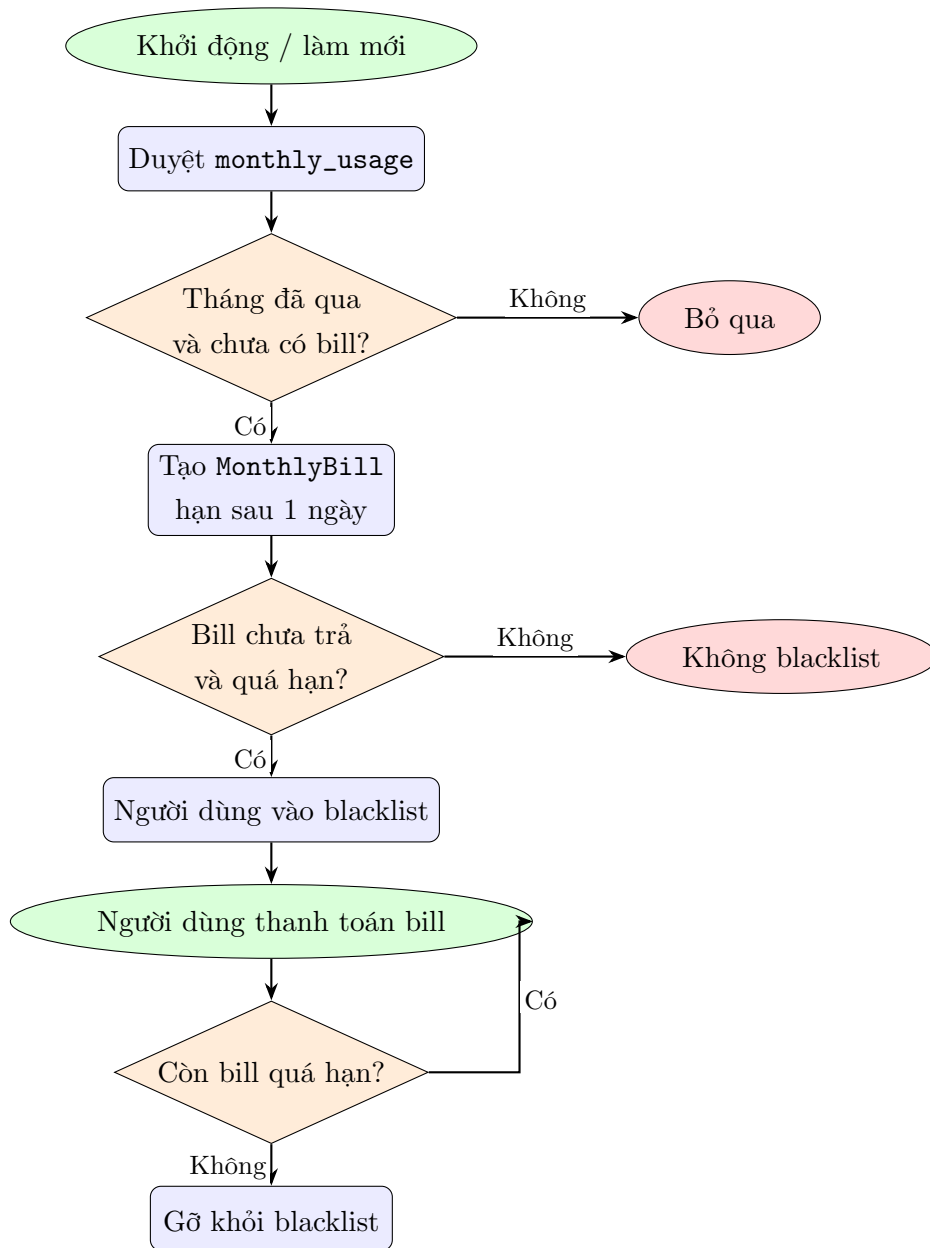
Hình 3.3: Luồng xử lý xe vào

3.5 Luồng xe ra



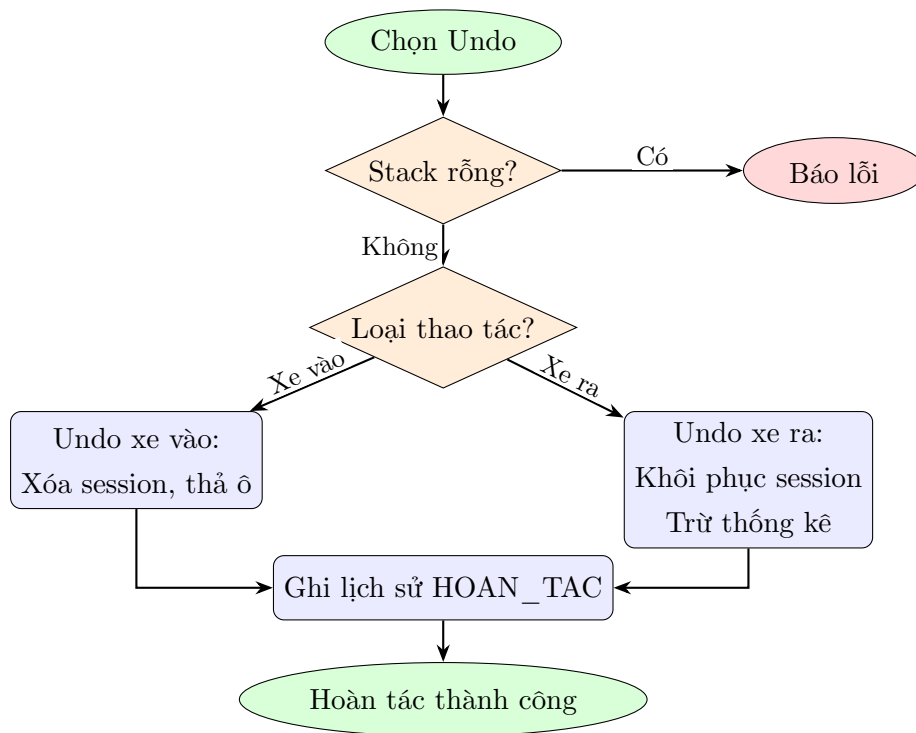
Hình 3.4: Luồng xử lý xe ra

3.6 Luồng hóa đơn và blacklist



Hình 3.5: Luồng hóa đơn và blacklist

3.7 Luồng undo



Hình 3.6: Luồng undo

Chương 4

Cấu trúc dữ liệu và giải thuật

4.1 Các cấu trúc dữ liệu

Bảng 4.1: Cấu trúc dữ liệu trong repository

File	Cài đặt thực tế	Vai trò thực tế	Độ phức tạp chính
LinkedList.hpp	Danh sách liên kết đôi; insertFront, insertBack, removeFront, removeBack, get, forEach.	Lưu lịch sử ParkingRecord; làm nền cho Stack và Queue.	Đầu/cuối: $O(1)$; tìm/get theo index: $O(n)$.
Stack.hpp	Stack bọc LinkedList, push/pop ở đầu danh sách.	Lưu AdminAction để undo xe vào/ra.	Push/pop/top: $O(1)$.
Queue.hpp	Queue bọc LinkedList, enqueue cuối, dequeue đầu.	Dùng trong BFS của ParkingMap::runBfs().	Enqueue/dequeue: $O(1)$.
BST.hpp	Cây nhị phân tìm kiếm cơ bản; insert, remove, search, duyệt.	Không dùng trực tiếp trong nghiệp vụ; tồn tại như cấu trúc môn học.	TB $O(\log n)$; xấu nhất $O(n)$.
AVL.hpp	Cây AVL; height, rotate left/right, rebalance, insert, remove, find, findByKey.	Là bucket trong HashTable; nền cho PriorityQueue.	Insert/remove/search: $O(\log n)$.
HashTable.hpp	Separate chaining bằng mảng bucket; mỗi bucket là AVL<pair<Key, Value>>; comparator chỉ so sánh key.	Lưu users, active sessions, monthly usage, monthly bills.	TB gần $O(1)$; trong bucket $O(\log k)$.
PriorityQueue.hpp	Hàng đợi ưu tiên bằng AVL; entry gồm value và order; phần tử trùng giá trị phân biệt bằng thứ tự chèn.	Không dùng trực tiếp trong nghiệp vụ; có trong thư viện CTDL.	Push/pop/top: $O(\log n)$.

4.2 Algorithm

lib/Algorithm.hpp cài các thuật toán: `linear_search`, `binary_search`, `selection_sort`, `insertion_sort`, `bubble_sort`, `heap_sort`, `merge_sort`, `quick_sort`. Các thuật toán này nằm trong thư viện chung; backend quản lý nhà xe không phụ thuộc trực tiếp vào chúng trong các luồng nghiệp vụ chính.

4.3 BFS tìm ô gần nhất

BFS được cài trong `ParkingMap::runBfs()`. Hệ thống chỉ duyệt qua ô lối đi (.) và cổng (E). Khoảng cách đến ô đỗ được tính bằng khoảng cách của ô đường kề bên cộng 1. Khi chọn ô, backend lọc theo role: giảng viên nhận T, sinh viên và khách nhận P. Nếu nhiều ô cùng khoảng cách, chọn `slotCode` nhỏ hơn (theo thứ tự từ điển).

```
BFS_FROM_ENTRY(map):
    dist = infinity for all cells
    dist[entry] = 0
    queue.enqueue(entry)
    while queue is not empty:
        current = queue.front()
        queue.dequeue()
        for each neighbor in 4 directions:
            if neighbor is ROAD or ENTRY and dist[neighbor] ==
              infinity:
                dist[neighbor] = dist[current] + 1
                queue.enqueue(neighbor)

FIND_SLOT(role):
    best = none
    for each slot s:
        if s.occupied: continue
        if s does not accept role: continue
        if distToSlot(s) == infinity: continue
        if best == none
            or distToSlot(s) < distToSlot(best)
            or (same distance and s.slotCode < best.slotCode):
            best = s
    return best
```

Listing 4.1: Mã giả BFS tìm ô gần nhất

4.4 Bảng độ phức tạp

Bảng 4.2: Độ phức tạp các thao tác nghiệp vụ chính

Thao tác	Thành phần chi phối	Độ phức tạp
Khởi tạo map	Đọc $R \times C$, BFS, xây danh sách slot	$O(RC + S)$
Tìm ô gần nhất	Duyệt toàn bộ slot đã xây	$O(S)$
Tra cứu user	HashTable bucket AVL	TB gần $O(1)$
Đăng ký user	Tra ID + duyệt kiểm tra trùng biển số	$O(U)$
Xe vào	Tra user, kiểm blacklist, tìm slot, ghi file	$O(B + S + A)$
Xe ra	Tra session, cập nhật usage, ghi file	$O(A + U + M)$
Duyệt lịch sử cá nhân	Duyệt <code>LinkedList</code> lịch sử	$O(H)$
Tạo hóa đơn tháng	Duyệt <code>monthlyUsage</code>	$O(M)$
Undo xe vào/xe ra	Pop stack, thao tác trên map + file	$O(1) + \text{I/O}$

Ký hiệu: R, C = chiều bản đồ; S = số ô đỗ; U = số user; B = số blacklist entries; A = số active sessions; M = số monthly usage records; H = số bản ghi lịch sử.

Chương 5

Thiết kế và lưu trữ dữ liệu

5.1 Nguyên tắc chung

Các file dữ liệu là UTF-8, mỗi dòng là một bản ghi, các trường phân tách bằng |. Loader có xử lý CRLF bằng cách bỏ \r. Một số loader bỏ qua dòng lỗi thay vì dừng toàn bộ chương trình. map.txt giữ bản đồ gốc; trạng thái X được tái tạo từ active_sessions.txt khi khởi động. Ghi file dùng cơ chế file tạm rồi rename để giảm nguy cơ hỏng dữ liệu.

5.2 DanhSachSV.txt

Bảng 5.1: Định dạng DanhSachSV.txt (Bảng 5.1)

Trường	Kiểu	Ý nghĩa
id	Chuỗi	Mã người dùng.
fullName	Chuỗi UTF-8	Họ tên.
licensePlate	Chuỗi	Biển số.
role	Số nguyên	0 sinh viên, 1 giảng viên, 2 khách.
totalParkingUnits	Số nguyên dài	Tổng lượt đã hoàn thành. Có thể thiếu ở format cũ (mặc định 0).

Ví dụ: 25120277 | Ngô Nguyễn Gia An | 59A-12345 | 0 | 10

5.3 map.txt

Dòng đầu gồm số dòng và số cột. Các dòng tiếp theo chứa ma trận ký tự: # tường, . lối đi, E cổng, P ô sinh viên/khách, T ô giảng viên.

5.4 active_sessions.txt

Bảng 5.2: Định dạng active_sessions.txt (Bảng 5.2)

Trường	Kiểu	Ý nghĩa
sessionId	Chuỗi	Mã phiên gửi xe.
userId	Chuỗi	ID người dùng.
licensePlate	Chuỗi	Biển số.
slotId	Số nguyên	ID số của ô.
slotCode	Chuỗi	Mã ô hiển thị (vd. A101).
entryTimeEpoch	Unix time	Thời điểm vào để tính toán.
entryTimestamp	Chuỗi	Thời điểm vào dạng ISO.

Format legacy cũ cũng được đọc: `userId | slotId | licensePlate | entryTimestamp`.

5.5 history.txt

```
eventId | sessionId | userId | licensePlate | name | slotId | slotCode |  
action | eventTimeEpoch | timestamp | durationSeconds | chargedUnits |  
amount
```

action gồm VAO, RA, HOAN_TAC_VAO, HOAN_TAC_RA, THANH_TOAN. Loader cũng hỗ trợ format cũ 6 trường.

5.6 monthly_usage.txt, monthly_bills.txt, revenue.txt

Bảng 5.3: Các file thống kê và hóa đơn (Bảng 5.3)

File	Định dạng
monthly_usage.txt	<code>userId year month parkingUnits amount</code>
monthly_bills.txt	<code>billId userId year month parkingUnits baseAmount createdAtEpoch dueEpoch paid paidAtEpoch</code>
revenue.txt	Dạng day month year amount count; định dạng kiểu cũ (legacy) và không phải là nguồn sự thật chính.

Chương 6

Cài đặt và hướng dẫn chạy

6.1 Linux TUI

Theo Makefile, build và chạy:

```
make tui
./parking_tui
# Co the truyen thu muc du lieu:
./parking_tui parking_test_data
```

Tham số thư mục được đọc trong `main(int argc, char* argv[])`; nếu không truyền thì dùng thư mục hiện tại.

6.2 Windows TUI

Theo build.bat:

```
build.bat tui
build.bat run-tui
```

Lệnh build trực tiếp tương ứng:

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic \
    -finput-charset=UTF-8 -fexec-charset=UTF-8 \
    -I.\app -I.\lib app\main_tui.cpp -o parking_tui.exe
```

6.3 Windows GUI

Theo build.bat:

```
build.bat gui
build.bat run-gui
```

GUI Win32 link các thư viện `gdi32`, `comctl32`, `comdlg32`, `shell32` và dùng tùy chọn `-mwindows`. GUI cũng nhận thư mục dữ liệu làm tham số đầu tiên.

6.4 Chạy kiểm thử backend

```
make test  
# Ket qua mong doi: backend tests passed
```

Chương 7

Giao diện và chức năng

Bảng 7.1: Giao diện và backend liên quan

Màn hình	Mục đích/luồng	Backend gọi	File thay đổi
Tổng quan	Hiển thị số user, xe đang gửi, lịch sử, hóa đơn quá hạn, doanh thu.	<code>processMonthlyBilling()</code> , <code>getStats()</code> , <code>overdueAmount()</code>	Có thể ghi <code>monthly_bills.txt</code> .
Xe vào	Nhập ID, nếu chưa đăng ký TUI có thể chuyển sang đăng ký; ghi nhận xe vào.	<code>vehicleEntry(...)</code>	<code>active_sessions.txt</code> , <code>history.txt</code> .
Xe ra	Nhập ID lần 1 xem vị trí, nhập lại ID xác nhận.	<code>previewVehicleExit(...)</code> , <code>vehicleExit(...)</code>	<code>DanhSachSV.txt</code> , <code>active_sessions.txt</code> , <code>monthly_usage.txt</code> , <code>history.txt</code> .
Đăng ký	Nhập ID, họ tên, biển số, phân loại.	<code>registerUser(...)</code>	<code>DanhSachSV.txt</code> .
Tra cứu	Tìm user, trạng thái đang gửi, blacklist, nợ.	<code>findUser(...)</code> , <code>findActiveSession(...)</code> , <code>outstandingAmount()</code>	Không đổi dữ liệu.
Danh sách người dùng	Duyệt toàn bộ user.	<code>forEachUser()</code> , <code>getUserList()</code>	Không đổi dữ liệu.
Xe đang gửi	Duyệt session, tìm biển số.	<code>forEachActiveSession()</code> , <code>searchByPlate()</code>	Không đổi dữ liệu.
Bản đồ	Hiển thị map với X cho ô đã có xe; cuộn bằng phím mũi tên.	<code>getMapLines()</code> , <code>loadMap(...)</code>	Không đổi dữ liệu.
Lịch sử cá nhân	Nhập ID và lọc lịch sử theo user.	<code>forEachPersonalHistory(...)</code>	Không đổi dữ liệu.
Hóa đơn	Xem bill theo user, nhập bill ID để trả.	<code>forEachBill(...)</code> , <code>payBill(...)</code>	<code>monthly_bills.txt</code> , <code>history.txt</code> .
Blacklist	Liệt kê user có bill quá hạn.	<code>forEachBlacklisted()</code> , <code>getBlacklist()</code>	Có thể ghi <code>monthly_bills.txt</code> .
Thống kê tháng	Nhập năm/tháng, xem lượt và tiền.	<code>forEachMonthlyUsage(...)</code>	Không đổi dữ liệu.

Bảng 7.1 (tiếp theo)

Màn hình	Mục đích/luồng	Backend gọi	File thay đổi
Undo	Hoàn tác thao tác xe vào/ra gần nhất.	<code>undoLast(...)</code>	Tùy loại: <code>active_sessions.txt</code> , <code>monthly_usage.txt</code> , <code>DanhSachSV.txt</code> , <code>history.txt</code> .

Chương 8

Kiểm thử

8.1 Kết quả build

Bảng 8.1: Kết quả build

Nền tảng	Target	Trạng thái	Warning	Ghi chú
Linux	make test	PASS	Không có trong output đã chạy	Đã build <code>backend_tests</code> và chạy thành công.
Linux	make tui	PASS	Chưa xác minh lại	Make báo target đã cập nhật.
Windows	TUI	NOT TESTED	NOT TESTED	Chỉ kiểm tra tính <code>build.bat</code> .
Windows	GUI Win32	NOT TESTED	NOT TESTED	Chỉ kiểm tra tính mã nguồn và lệnh build.

8.2 Bảng test case

Bảng 8.2: Test case chi tiết

Mã	Chức năng	Điều kiện đầu	Input	Mong đợi	Thực tế	Trạng thái
U01	Load user 4 trường	File có <code>OLD1 ... 0</code>	OLD1	User tồn tại, lượt = 0	Đúng (test tự động)	PASS
U02	Load user 5 trường	File có <code>NEW1</code>	<code>... 1 7</code>	Lượt = 7	Đúng (test tự động)	PASS
U03	Đăng ký mới	ID chưa có	NEW2	Thành công	Đúng (test tự động)	PASS

Bảng 8.2 (tiếp theo)

Mã	Chức năng	Điều kiện đầu	Input	Mong đợi	Thực tế	Trạng thái
U04	Trùng ID	OLD1 đã có	OLD1	Từ chối	Đúng (test tự động)	PASS
U05	Trùng biển số	Biển số đã có	AA-001	Từ chối	Đúng (test tự động)	PASS
U06	Role không hợp lệ	Dòng role 9	BAD1	Bỏ qua user lỗi	Đúng (test tự động)	PASS
U07	Tra cứu không tồn tại	Backend đã khởi tạo	ID lạ	Báo không tìm thấy	Chưa xác minh	NOT TEST
E01	Xe vào sinh viên	Có ô P trống	S1	Nhận ô P	Đúng (test tự động)	PASS
E02	Xe vào giảng viên	Có ô T trống	T1	Nhận ô T	Đúng (test tự động)	PASS
E03	Xe vào khách	Có ô P trống	G1	Nhận ô P	Đúng (test tự động)	PASS
E04	ID chưa tồn tại	Không có user	ID lạ	Từ chối	Đúng (test tự động)	PASS
E05	Xe đã trong bãi	User có session	S1	Từ chối lần 2	Đúng (test tự động)	PASS
E06	Người bị blacklist	Có bill quá hạn	B1	Từ chối	Đúng (test tự động)	PASS
E07	Hết chỗ phù hợp	Map chỉ có 1 ô P	S2	Từ chối	Đúng (test tự động)	PASS
X01	Tìm đúng vị trí	User có session	ID đang gửi	Trả slotCode đúng	Chưa xác minh	NOT TEST

Bảng 8.2 (tiếp theo)

Mã	Chức năng	Điều kiện đầu	Input	Mong đợi	Thực tế	Trạng thái
X02	Xác nhận sai ID	User có session	WRONG	Từ chối	Đúng (test tự động)	PASS
X03	Dưới 10 giờ	User có session	< 36000s	1 lượt	Chưa xác minh riêng	NOT TEST
X04	Đúng 10 giờ	User S1	36000s	1 lượt, 3000 đ	Đúng (test tự động)	PASS
X05	Trên 10 giờ	User G1	36001s	2 lượt, 10000 đ	Đúng (test tự động)	PASS
X06	Trên 20 giờ	User T1	72001s	3 lượt, 15000 đ	Đúng (test tự động)	PASS
X07	Khôi phục P/T	Sau xe ra	slot cũ	Status empty, baseType giữ	Kiểm P; T chưa riêng	NOT TEST
H01	Lịch sử xe vào	Xe vào thành công	ID hợp lệ	Có VAO	Bao phủ gián tiếp	PASS
H02	Lịch sử xe ra	Xe ra thành công	ID hợp lệ	Có RA	Bao phủ gián tiếp	PASS
H03	Lịch sử thanh toán	Có bill	bill ID	Có THANH_TOAN	Bao phủ gián tiếp	PASS
H04	Lịch sử undo	Có undo	Undo	Có HOAN_TAC_*	Bao phủ gián tiếp	PASS
H05	Lịch sử cá nhân	Nhiều record	userId	Chỉ record của user	Chưa xác minh	NOT TEST
B01	Đã thanh toán	Bill tồn tại	bill ID	paid = 1	Đúng (test tự động)	PASS
B02	Chưa đến hạn	Bill chưa quá hạn	now trước hạn	Không blacklist	Chưa xác minh	NOT TEST

Bảng 8.2 (tiếp theo)

Mã	Chức năng	Điều kiện đầu	Input	Mong đợi	Thực tế	Trạng thái
B03	Quá hạn 1 ngày	Bill quá hạn	now sau hạn	10000/ngày	Chưa có assert riêng	NOT TEST
B04	Quá hạn nhiều ngày	Bill tháng 5, now tháng 7	B1	paid > base	Đúng (test tự động)	PASS
B05	Nhiều bill quá hạn	User nhiều bill	userId	Cộng nhiều bill	Chưa xác minh	NOT TEST
B06	Thanh toán gỡ blacklist	User B1	bill ID	Không blacklist sau trả	Đúng (test tự động)	PASS
D01	Undo xe vào	Vừa xe vào	Undo	Xóa active session	Đúng (test tự động)	PASS
D02	Undo xe ra	Vừa xe ra	Undo	Khôi phục session	Đúng (test tự động)	PASS
D03	Ô cũ bị chiếm	Sau xe ra, chiếm slot	Undo	Từ chối	Đúng (test tự động)	PASS
D04	Stack rỗng	Không có undo	Undo	Từ chối	Đúng (test tự động)	PASS
P01	Xe vào rồi reload	Có active session	P1	Session còn tồn tại	Đúng (test tự động)	PASS
P02	Xe ra rồi reload	Sau xe ra	Reload	Không còn session	Chưa xác minh	NOT TEST
P03	Thanh toán rồi reload	Sau trả bill	Reload	Bill vẫn paid	Chưa xác minh	NOT TEST
C01	Build Linux TUI	Repo hiện tại	<code>make tui</code>	Target OK	Make báo đã cập nhật	PASS

Bảng 8.2 (tiếp theo)

Mã	Chức năng	Điều kiện đầu	Input	Mong đợi	Thực tế	Trạng thái
C02	Build Windows TUI	Windows/MSYS2	Build.bat tui	Build OK	NOT TESTED	NOT TEST
C03	Build Windows GUI	Windows/MSYS2	Build.bat gui	Build OK	NOT TESTED	NOT TEST
C04	UTF-8 tiếng Việt	Terminal	Chuỗi tiếng Việt	Không lỗi mã hóa	Chưa xác minh runtime	NOT TEST
C05	LF và CRLF	File có CRLF	Dòng CRLF	Loader bỏ \r	Có xử lý trong code	NOT TEST

Chương 9

Đánh giá và hướng phát triển

9.1 Những phần đã hoàn thành

Project đã hoàn thành backend quản lý người dùng, xe vào, xe ra, bản đồ (BFS), thống kê tháng, hóa đơn, blacklist, thanh toán, lịch sử và undo xe vào/ra. Hai giao diện TUI và GUI cùng sử dụng backend chung. Các cấu trúc dữ liệu tự cài đặt được dùng trong backend, đặc biệt `HashTable`, `LinkedList`, `Stack` và `Queue`.

9.2 Điểm mạnh

- Backend dùng chung giúp TUI và GUI nhất quán định dạng dữ liệu.
- Bản đồ không ghi đè trạng thái **X**; trạng thái được tái tạo từ active sessions nên dễ bảo toàn map gốc.
- Ghi một số file bằng cơ chế file tạm rồi **rename**, giảm nguy cơ hỏng dữ liệu khi ghi thất bại.
- Có test backend tự động trong `tests/backend_tests.cpp` bao phủ các tình huống chính: đăng ký, xe vào/ra, phí, blacklist, thanh toán, undo và persistence.
- Hỗ trợ cả format dữ liệu mới và legacy; loader bỏ qua dòng lỗi thay vì crash.

9.3 Hạn chế

- Undo chỉ tồn tại trong phiên chạy hiện tại, không lưu stack undo qua lần khởi động lại.
- GUI Win32 dùng tọa độ cố định, phù hợp desktop hơn là màn hình nhỏ.
- Chưa xác minh build và runtime GUI Windows trong môi trường hiện tại.
- Dữ liệu text phù hợp đồ án nhỏ nhưng chưa có ràng buộc giao dịch (transaction) như cơ sở dữ liệu.
- Một số test case (lịch sử cá nhân, reload sau xe ra) chưa có assert riêng.

9.4 Hướng phát triển

- Chuyển lưu trữ sang SQLite hoặc cơ sở dữ liệu nhẹ để có transaction safety.
- Tích hợp RFID/camera nhận diện biển số tự động.
- Cảm biến ô nhiễm thời gian thực cập nhật trạng thái map.
- Phân quyền quản trị (admin/operator).
- Báo cáo doanh thu nâng cao, dashboard theo thời gian thực.
- Đồng bộ nhiều máy trạm (client-server).
- Lưu stack undo ra đĩa để hỗ trợ undo xuyên phiên.

Kết luận

Đồ án đã xây dựng được ứng dụng quản lý nhà xe bằng C++ với backend tương đối đầy đủ, có hai giao diện TUI và Win32 GUI, lưu trữ file UTF-8 và sử dụng các cấu trúc dữ liệu tự cài đặt. Các chức năng cốt lõi như đăng ký, xe vào, xe ra, tìm ô gần nhất (BFS), hóa đơn, blacklist, thanh toán và undo đã được tổ chức rõ trong **ParkingSystem** và **ParkingMap**. Hạn chế chính nằm ở lưu trữ dạng file, undo chưa bền vững sau khi tắt chương trình và GUI Windows chưa được xác minh runtime trong môi trường hiện tại.

Tài liệu tham khảo

- [1] cppreference.com, *C++ Standard Library Reference*, truy cập 2025, cho các khái niệm `std::time_t`, `std::fstream`, `std::vector`, `std::string`.
<https://en.cppreference.com>
- [2] Microsoft Learn, *Win32 API Documentation*, tham khảo cho `CreateWindowExW`, `MessageBoxW`, `ListView`, `TabCtrl`.
<https://learn.microsoft.com/en-us/windows/win32/api/>
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009. Tham khảo BFS, cây AVL và độ phức tạp thuật toán.

Chương A

Cấu trúc repository đã kiểm tra

```
app/  
  main.cpp  
  main_tui.cpp  
  models.hpp  
  models_tui.hpp  
  ParkingMap.hpp  
  ParkingMap_tui.hpp  
  ParkingSystem.hpp  
  ParkingSystem_tui.hpp  
  Terminal.hpp  
lib/  
  Algorithm.hpp  
  AVL.hpp  
  BST.hpp  
  HashTable.hpp  
  LinkedList.hpp  
  PriorityQueue.hpp  
  Queue.hpp  
  Stack.hpp  
tests/  
  backend_tests.cpp  
Makefile  
build.bat  
README.md  
DanhSachSV.txt  
map.txt  
active_sessions.txt  
history.txt  
monthly_usage.txt  
monthly_bills.txt  
revenue.txt
```

Chương B

Lệnh kiểm tra đã chạy trong môi trường hiện tại

```
make test
# Ket qua chinh:
# [OK] Built backend tests: backend_tests
# backend tests passed

make tui
# Ket qua chinh:
# make: Nothing to be done for 'tui'.
```